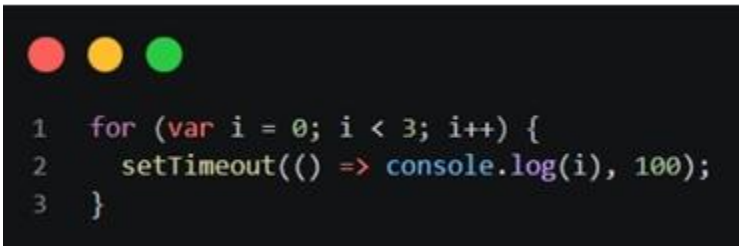
Nama	: Muhammad Habib Fadhilah
Nomor Mentees	: M2-BE-13

LAPORAN TUGAS PERTEMUAN 3 BE METRO INSTITUTE

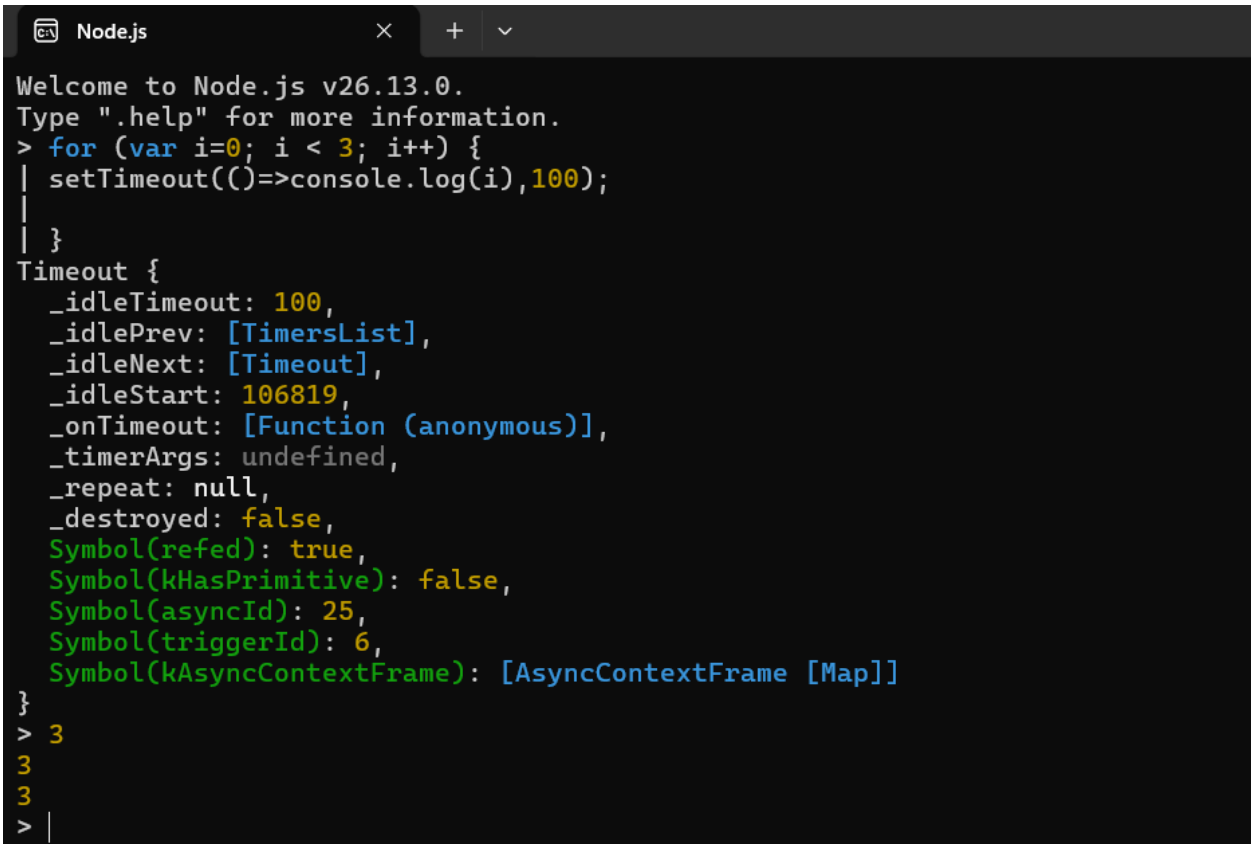
Soal 1



```
1  for (var i = 0; i < 3; i++) {
2    setTimeout(() => console.log(i), 100);
3  }
```

Hipotesis: Output muncul mungkin 0, 1, 2. Kode menjalankan perulangan dari 0 hingga 2, karena nilai i < 3.

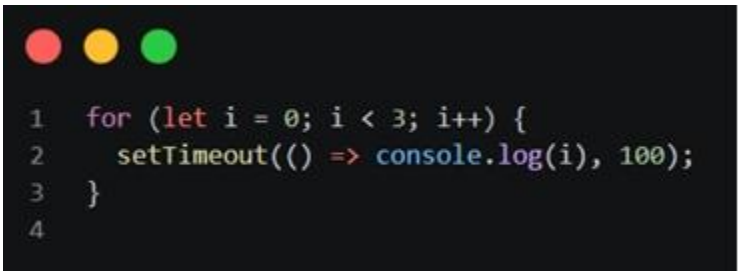
Jawaban:



```
Node.js
Welcome to Node.js v26.13.0.
Type ".help" for more information.
> for (var i=0; i < 3; i++) {
| setTimeout(()=>console.log(i),100);
|
| }
Timeout {
  _idleTimeout: 100,
  _idlePrev: [TimersList],
  _idleNext: [Timeout],
  _idleStart: 106819,
  _onTimeout: [Function (anonymous)],
  _timerArgs: undefined,
  _repeat: null,
  _destroyed: false,
  Symbol(refed): true,
  Symbol(kHasPrimitive): false,
  Symbol(asyncId): 25,
  Symbol(triggerId): 6,
  Symbol(kAsyncContextFrame): [AsyncContextFrame [Map]]
}
> 3
3
3
>
```

Variabel i menggunakan var yang bersifat function scope sehingga seluruh iterasi berbagi satu nilai i yang sama di memori. Ketika perulangan synchronous selesai, nilai i sudah bertambah menjadi 3. Oleh karena itu, ketiga callback setTimeout yang bersifat asynchronous baru dieksekusi setelah 100ms dan sama-sama mencetak nilai akhir i tersebut, yaitu 3.

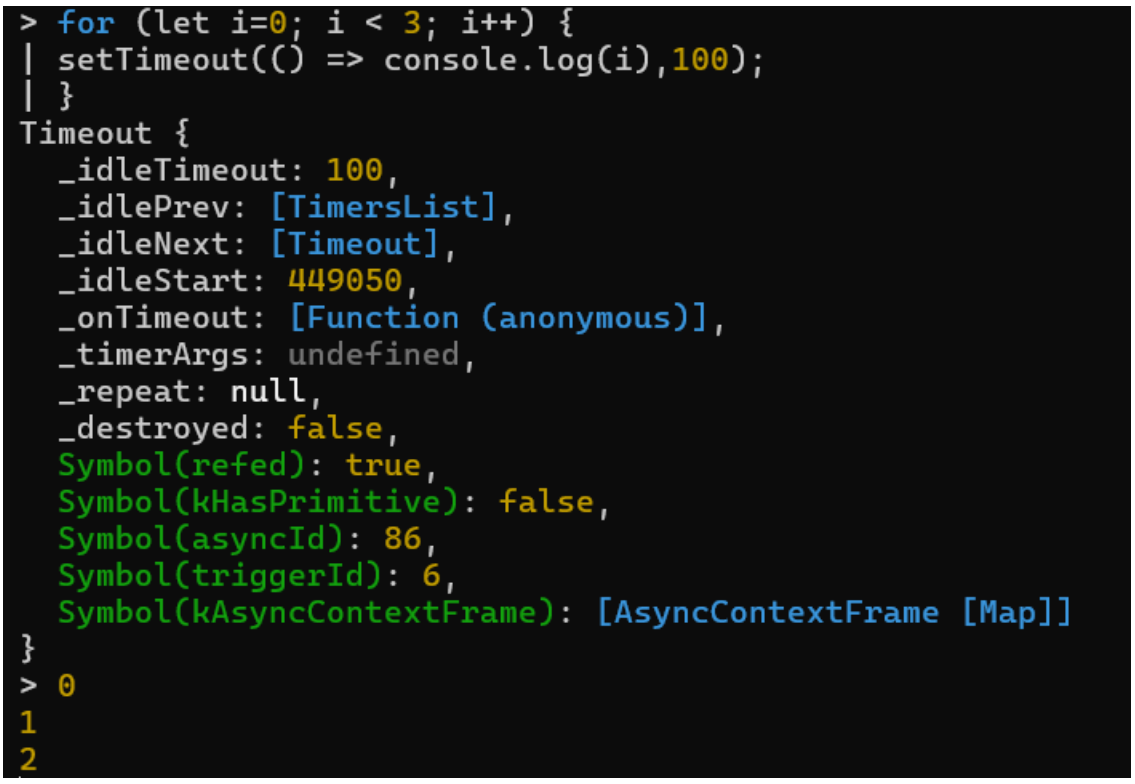
Soal 2



```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

Hipotesis: Output yang akan dihasilkan adalah angka 0, 1, dan 2. Hal ini karena variabel i menggunakan let yang memiliki block scope. Dengan demikian, setiap iterasi perulangan memiliki nilai variabel tersendiri yang dapat diakses oleh fungsi setTimeout().

Jawaban:



```
> for (let i=0; i < 3; i++) {
| setTimeout(() => console.log(i),100);
| }
Timeout {
  _idleTimeout: 100,
  _idlePrev: [TimersList],
  _idleNext: [Timeout],
  _idleStart: 449050,
  _onTimeout: [Function (anonymous)],
  _timerArgs: undefined,
  _repeat: null,
  _destroyed: false,
  Symbol(refed): true,
  Symbol(kHasPrimitive): false,
  Symbol(asyncId): 86,
  Symbol(triggerId): 6,
  Symbol(kAsyncContextFrame): [AsyncContextFrame [Map]]
}
> 0
1
2
```

Penggunaan let menciptakan block scope baru pada setiap iterasi perulangan, sehingga setiap putaran menyimpan nilai i tersendiri secara independen di memori. Ketika callback setTimeout dipanggil secara asynchronous setelah 100ms, masing-masing fungsi membaca dan mencetak nilai i spesifik yang ditangkap dari block scope iterasinya masing-masing (0, 1, dan 2).

Soal 3

```
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), i * 100);
}
```

Hipotesis: Output yang akan dihasilkan adalah angka 4 sebanyak tiga kali. Namun, setiap output akan muncul dengan jeda waktu yang berbeda, yaitu 100 milidetik, 200 milidetik, dan 300 milidetik. Hal ini karena variabel i menggunakan var, sehingga ketiga fungsi setTimeout() mengakses variabel yang sama.

Jawaban:

```
> for (var i = 1; i <= 3; i++) {
| setTimeout(()=>console.log(i),i*100);
|
| }
Timeout {
  _idleTimeout: 300,
  _idlePrev: [TimersList],
  _idleNext: [TimersList],
  _idleStart: 554712,
  _onTimeout: [Function (anonymous)],
  _timerArgs: undefined,
  _repeat: null,
  _destroyed: false,
  Symbol(refed): true,
  Symbol(kHasPrimitive): false,
  Symbol(asyncId): 103,
  Symbol(triggerId): 6,
  Symbol(kAsyncContextFrame): [AsyncContextFrame [Map]]
}
> 4
4
4
```

Karena deklarasi var bersifat function scope, perulangan utama menyelesaikan eksekusinya terlebih dahulu hingga nilai i mencapai 3. Callback setTimeout pertama (delay 100ms) dan kedua (delay 200ms) baru dieksekusi secara asynchronous oleh event loop setelah loop usai, sehingga keduanya membaca dan mencetak referensi variabel i akhir yang bernilai 3.

Soal 4

```
for (var i = 0; i < 3; i++) {
  let j = i;
  setTimeout(() => console.log(i, j), 0);
}
```

Hipotesis: Menurut saya, output yang akan dihasilkan adalah angka 3 yang dipasangkan dengan angka 0, kemudian angka 3 dengan angka 1, dan terakhir angka 3 dengan angka 2, karena variabel i menggunakan var, sedangkan variabel j menggunakan let. Variabel i akan memiliki nilai akhir yang sama ketika fungsi dijalankan, sementara variabel j mempertahankan nilai masing-masing iterasi.

Jawaban:

```
> for (var i=0; i < 3; i++) {
| let j = i;
| setTimeout(() => console.log(i,j),0);
| }
Timeout {
  _idleTimeout: 1,
  _idlePrev: [TimersList],
  _idleNext: [Timeout],
  _idleStart: 661012,
  _onTimeout: [Function (anonymous)],
  _timerArgs: undefined,
  _repeat: null,
  _destroyed: false,
  Symbol(refed): true,
  Symbol(kHasPrimitive): false,
  Symbol(asyncId): 180,
  Symbol(triggerId): 6,
  Symbol(kAsyncContextFrame): [AsyncContextFrame [Map]]
}
> 3 0
3 1
3 2
```

Variabel i (var) membagikan satu nilai akhir yaitu 3 setelah perulangan usai, sedangkan variable (let) dibuat baru pada block scope di setiap iterasi untuk menyimpan salinan nilai i saat itu (0, 1, 2). Walaupun delay setTimeout diset 0ms, callback tetap diproses secara asynchronous setelah alur utama selesai, sehingga mencetak nilai i yang bernilai 3 berdampingan dengan nilai j masing-masing iterasi.

Soal 5

```
1  for (var i = 0; i < 3; i++) {
2    setTimeout(() => console.log('di dalam:', i), 0);
3  }
4  console.log('di luar:', i);
```

Hipotesis: Teks 'di luar:' dicetak terlebih dahulu sebelum 3 baris teks 'di dalam:'

Jawaban:

```
> for (var i=0; i < 3; i++) {
| setTimeout(()=>console.log('di dalam:',i), 0);
|
| console.log('di luar:', i);
| }
di luar: 0
di luar: 1
di luar: 2
undefined
> di dalam: 3
di dalam: 3
di dalam: 3
```

Kode synchronous console.log('di luar:', i) langsung dieksekusi pada alur utama (Call Stack) segera setelah perulangan selesai dengan nilai i = 3. Sementara itu, ketiga callback setTimeout (meskipun delay 0ms) dimasukkan ke dalam antrean asynchronous (Callback Queue) dan baru dieksekusi setelah Call Stack kosong, sehingga pesan 'di dalam:' dicetak belakangan dengan merujuk nilai i akhir yaitu 3.